

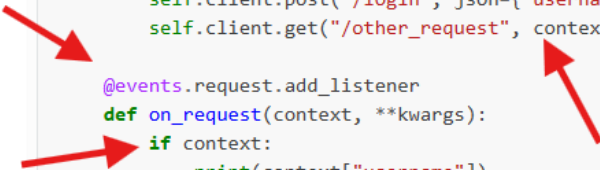
Request context

The `request event` has a context parameter that enable you to pass data *about* the request (things like username, tags etc). It can be set directly in the call to the request method or at the User level, by overriding the `User.context()` method.

Context from request method:

```
class MyUser(HttpUser):
    @task
    def t(self):
        self.client.post("/login", json={"username": "foo"})
        self.client.get("/other_request", context={"username": "foo"})

    @events.request.add_listener
    def on_request(context, **kwargs):
        if context:
            print(context["username"])
```

A diagram with three red arrows. One arrow points from the `context` parameter in the `self.client.get` call to the `context` parameter in the `on_request` function definition. A second arrow points from the `on_request` function definition to the `context` parameter in the `print` statement. A third arrow points from the `context` parameter in the `on_request` function definition to the `context` parameter in the `self.client.get` call.

=====

Context from a value in the response, using `catch_response`:

```
with self.client.get("/", catch_response=True) as resp:
    resp.request_meta["context"]["requestId"] = resp.json()["requestId"]
```

! Note

Request context doesn't change how Locust's regular statistics are calculated.

=====

Adding Web Routes

Locust uses Flask to serve the web UI and therefore it is easy to add web end-points to the web UI. By listening to the `init` event, we can retrieve a reference to the Flask app instance and use that to set up a new route:

```
from locust import events

@events.init.add_listener
def on_locust_init(environment, **kw):
    @environment.web_ui.app.route("/added_page")
    def my_added_page():
        return "Another page"
```

A red arrow points from the `on_locust_init` function definition to the `my_added_page` function definition.

=====

Run a background greenlet

Because a locust file is “just code”, there is nothing preventing you from spawning your own greenlet to run in parallel with your actual load/Users.

For example, you can monitor the fail ratio of your test and stop the run if it goes above some threshold:

```
import gevent
from locust import events
from locust.runners import STATE_STOPPING, STATE_STOPPED, STATE_CLEANUP, MasterRunner, LocalRunner

def checker(environment):
    while not environment.runner.state in [STATE_STOPPING, STATE_STOPPED, STATE_CLEANUP]:
        time.sleep(1)
        if environment.runner.stats.total.fail_ratio > 0.2:
            print(f"fail ratio was {environment.runner.stats.total.fail_ratio}, quitting")
            environment.runner.quit()
            return

@events.init.add_listener
def on_locust_init(environment, **kwargs):
    # dont run this on workers, we only care about the aggregated numbers
    if isinstance(environment.runner, MasterRunner) or isinstance(environment.runner, LocalRunner):
        gevent.spawn(checker, environment)
```

=====

Per-request CSV logging

The built-in `--csv` flag exports only **aggregated** statistics. For post-run analysis that requires individual data-points (outlier detection, per-endpoint latency histograms, failure forensics, Pandas/BI pipelines), use `CsvRequestLogger`:

```
from locust import HttpUser, task, events
from locust.contrib.csv_request_logger import CsvRequestLogger

logger = CsvRequestLogger("results/requests.csv")

@events.init.add_listener
def on_locust_init(environment, **kwargs):
    logger.register(environment)

class MyUser(HttpUser):
    @task
    def index(self):
        self.client.get("/")
```

This writes one CSV row per completed request with columns: `timestamp`, `request_type`, `name`, `response_time_ms`, `response_length`, `status_code`, and `exception`. The logger works with any protocol (HTTP, WebSocket, gRPC, custom) and closes automatically on shutdown. See `locust.contrib.csv_request_logger` for full API documentation.

=====

Logging

Locust uses Python's [built in logging framework](#) for handling logging.

The default logging configuration that Locust applies, writes log messages directly to stderr. `--loglevel` and `--logfile` can be used to change the verbosity and/or make the log go to a file instead.

The default logging configuration installs handlers for the `root` logger as well as the `locust.*` loggers, so using the root logger in your own test scripts will put the message into the log file if `--logfile` is used:

```
import logging
logging.info("this log message will go wherever the other locust log messages go")
```

It's also possible to control the whole logging configuration in your own test scripts by using the `--skip-log-setup` option. You will then have to [configure the logging](#) yourself.

=====